Accelerating AES

Michael Benney (z5478530)
Daniel Craig (z5417681)
Aaron Nyholm (z5316510)
Xavier Herbert (z5478413)

Project Overview

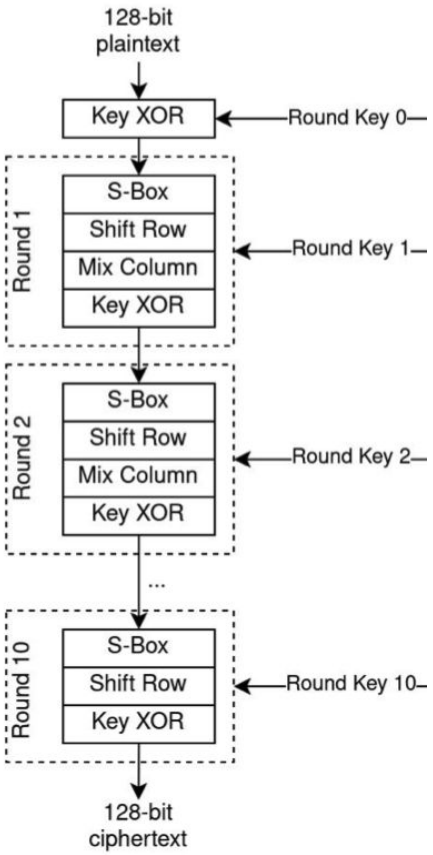
- Aims – we have focused on AES encryption in ECB mode with PKCS#7

Aims (from initial plan):

- Implement AES in hardware to allow for encryption offloading ✅
- Utilise parallelism of AES to provide encryption acceleration ✅
- Reduce bottlenecks experienced with offloading to provide a seamless acceleration ❓
- Achieve above goals without compromising security by exposing encryption key ❌❓

- Key Result: **73x** speedup from running AES on the KV260 ARM processor

AES Intro



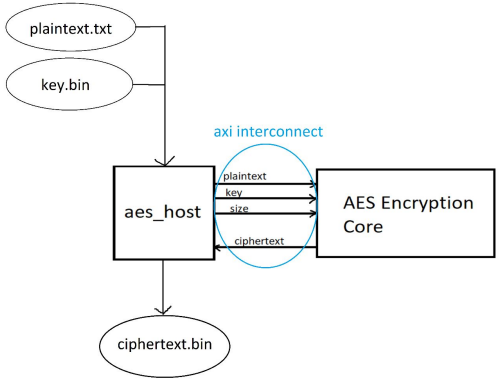

```c
#define NUM_ROUNDS 10
#define SIZE 128
#define SBOX_DIM = 16

D_TYPE round_keys[NUM_ROUNDS+1][SIZE] = { ... } // round keys for encryption
uint8_t rjindael_sbox[SBOX_DIM][SBOX_DIM] = { ... } // instantiated forward Rjindael's Sbox for sbox funct
void aes(D_TYPE plaintext[SIZE], D_TYPE ciphertext[SIZE]) {

    // key expansion
    key_expansion(round_keys);

    // define buffers
    D_TYPE key_xor_buf[SIZE], sbox_buf[SIZE], sr_buf[SIZE], mc_buf[SIZE];

    // Round 0
    key_xor(plaintext, round_keys[0], key_xor_buf);

    // Rounds 1 - 10
    for (int i = 1; i <= NUM_ROUNDS; i++) {
        // SBOX sub
        sbox(key_xor_buf, rjindael_sbox, sbox_buf);
        // shift row
        sr(sbox_buf, sr_buf);
        // if Round 1-9, Mix Cols, Else key_xor
        if (i < NUM_ROUNDS) {
            mc(sr_buf, mc_buf);
            key_xor(mc_buf, round_keys[i], key_xor_buf);
        } else {
            key_xor(sr_buf, round_keys[i], key_xor_buf);
        }
    }

    // output is in final buffer
    ciphertext = key_xor_buf;
}
```

Base platform

- We built upon a open-source reference C AES implementation
- Running the software on the board
- Running the reference code as HLS in hardware

```
void aes_encrypt(const uint8_t* plaintext, uint8_t* ciphertext, const uint8_t* key);
```

- **Verification**: Compared with plaintext, key and ciphertext pairs using C libraries



Initial Improvement Analysis

- Number of kernel accesses required
  - One kernel access encrypted 16 bytes of data
- Improved the HLS function API

```
void aes_encrypt(const uint8_t* plaintext, const uint32_t size, uint8_t* ciphertext,
    const uint8_t* key);
```

Acceleration changes

- Expanded input size to minimise kernel access overhead
- Removed the Key Pre-processing step
  - Happens in parallel to generate each round as needed
  - Only delays when it takes longer than sub bytes
  - AES allows for instant start with key format
- Converted Galois Multiplication to Lookup Table
  - Encoding needs Mul by 2 and 3 only
- Added pipelining for the encryption loop
  - Fully unrolled loops to allow for stages to happen in blocks

Results

- Software Unoptimised (on board) 0.127MB/s
- Hardware Unoptimised 0.312MB/s
- Flexible Input Size Unoptimised 2.5MB/s
- Flexible Input Size Optimised 9.29MB/s

Discussion – Design Evaluation

- Our design meets validation requirements.
- The optimised design has a **31x** speedup from the unoptimised design, and a **73x** speedup from running AES on the KV260 ARM processor.
- However, AES on our laptops is still faster than running AES on our optimised kernel: **8** ms vs **112** ms for a 1MB file.
- The runtime does not change significantly for a 1MB file between inputting 2^13 AES blocks and inputting 2^6.
    - Our design is successfully pipelining kernel accesses.
- There are still bottlenecks – **keygen** and **AXI writeresps** on each AES loop.

– II=16 for encryption loop, roundkeygen not unrolled

Discussion – Security Evaluation

- From a security point of view, there are several issues with the current design:
  - We have only implemented ECB mode, whereas CBC mode is far more secure.
  - The key is input over an AXI interface, leaves the design vulnerable to **hardware trojans**.
  - If we were to create a custom chip using this design, since all intermediate values are stored in registers, there is a vulnerability to **scan chain attacks**.
- Hence, we do **not** believe that we have succeeded in the aim of delivering a secure AES encryption offloader.

Conclusion

- AES is a simple enough algorithm, which made optimising it easier.
- The main issue we encountered was time and organisation.
- Given more time, we would:
  - Squeeze out the last bits of optimisation. Consider even moving to HDL.
  - Implement CBC mode AES for enhanced security, consider keeping the key inside the AES encryption module.
  - Implement a decryption core which the host can separately instantiate.

- Overall, we believe that given the speedups we achieved, we have largely met our aims.
- AI was used purely to help with testbench generation, all other components were painfully human generated.